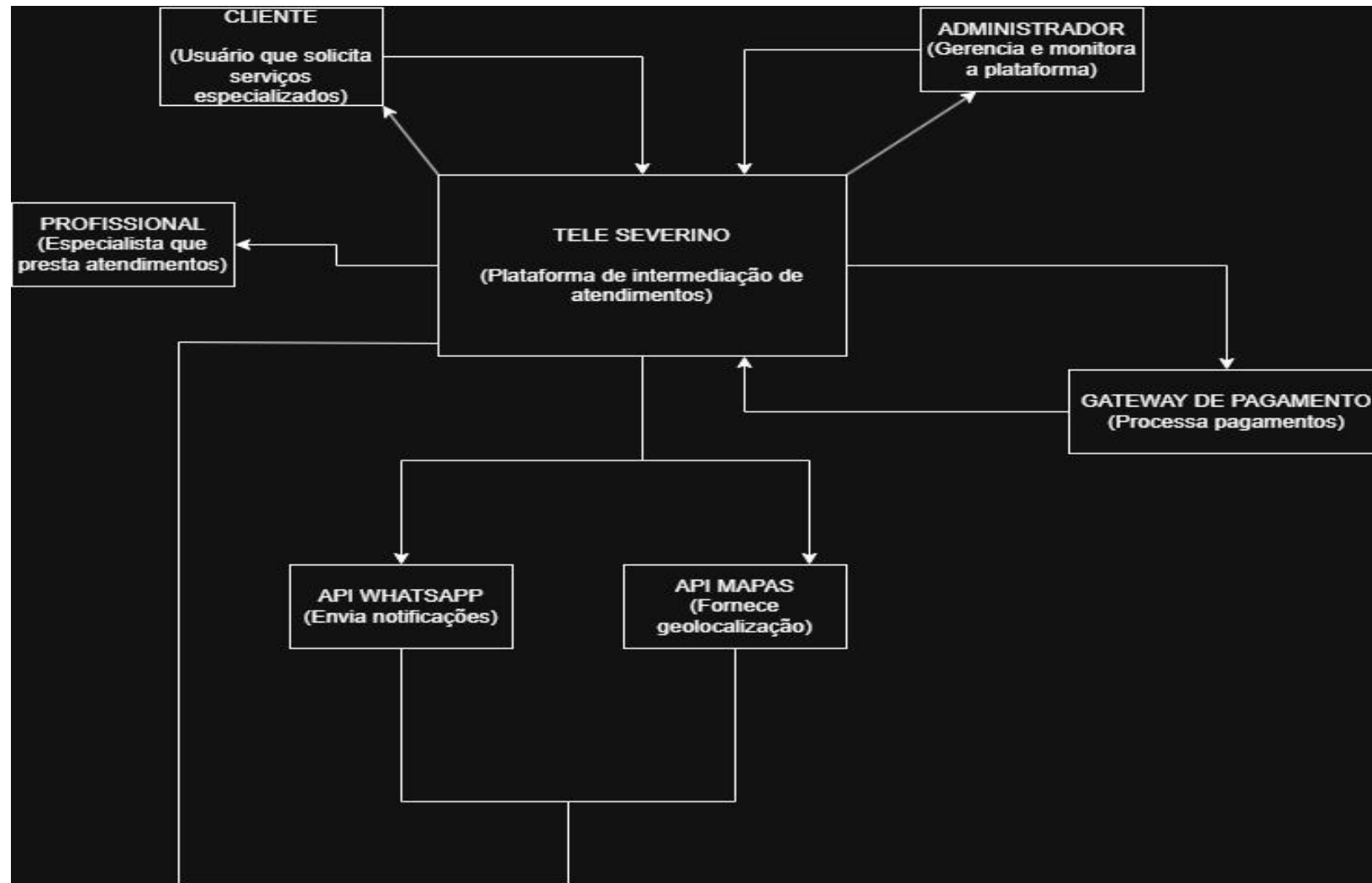


DOCUMENTAÇÃO DE ARQUITETURA DE SOFTWARE: MODELO C4PROJETO: Tele Severino

NÍVEL 1: DIAGRAMA DE CONTEXTO

O Diagrama de Contexto mostra até onde o Tele Severino atua e com quem ele se comunica. Nele, a plataforma é apresentada como o sistema central, conectando clientes, estabelecimentos parceiros e serviços externos, como APIs de pagamento e localização. O foco desse diagrama é deixar claro quais atores interagem com o sistema e quais informações entram e saem da plataforma, sem detalhar programação, banco de dados ou estrutura interna.

Figura 1: Diagrama de Contexto (C1) do Sistema Tele Severino. Código-fonte do Diagrama



%% Atores / Usuários

Cliente ["Cliente
<i> (Usuário que solicita serviços especializados) </i>"]

Admin ["Administrador
<i> (Gerência e monitora a plataforma) </i>"]

Profissional ["Profissional
<i> (Especialista que presta atendimentos) </i>"]

%% Sistema Central

Sistema ["TELE SEVERINO
<i> (Plataforma de intermediação de atendimentos) </i>"]

%% Sistemas Externos

Gateway ["GATEWAY DE PAGAMENTO
<i> (Processa pagamentos) </i>"]

Zap ["API WHATSAPP
<i> (Envia notificações) </i>"]

Mapas ["API MAPAS
<i> (Fornece geolocalização) </i>"]

%% Relacionamentos e Fluxos

Cliente -->|Solicita serviços e paga| Sistema

Sistema -->|Retorna informações e status| Cliente

Admin -->|Gerencia e monitora| Sistema

Sistema -->|Envia dados de auditoria e relatórios| Admin

Sistema -->|Direciona atendimentos| Profissional

Profissional -->|Aceita e realiza atendimentos| Sistema

Sistema -->|Envia dados da cobrança| Gateway

Gateway -->|Confirma status do pagamento| Sistema

Sistema -->|Dispara mensagens automáticas| Zap

Sistema -->|Consome rotas e coordenadas| Mapas

%% Estilização Padrão C4 (Contexto)

style Sistema fill: #1168bd, stroke: #0b4c8c, stroke-width:2px, color: #fff

style Cliente fill: #08427b, stroke: #052e56, stroke-width:2px, color: #fff

style Admin fill: #08427b, stroke: #052e56, stroke-width:2px, color: #fff

style Profissional fill: #08427b, stroke: #052e56, stroke-width:2px, color: #fff

style Gateway fill: #999999, stroke: #666666, stroke-width:2px, color: #fff

style Zap fill: #999999, stroke: #666666, stroke-width:2px, color: #fff

style Mapa's fill: #999999, stroke: #666666, stroke-width:2px, color: #fff

Relatório Técnico do Contexto: Atores do Sistema:

Cliente: Usuário final que acessa a plataforma em busca de um atendimento remoto especializado (áudio/vídeo). É responsável por iniciar a chamada e realizar o pagamento.

Profissional: Especialista técnico cadastrado que disponibiliza seu tempo na plataforma para realizar os atendimentos e recebe os repasses financeiros proporcionalmente aos minutos trabalhados.

Administrador: Usuário interno que monitora a saúde da plataforma, extrai relatórios gerenciais e resolve disputas ou suporte.

Sistema Central (Tele Severino): A plataforma central que gerencia as conexões, calcula a tarifação do atendimento por minuto, controla os perfis de usuários e orquestra as regras gerais do negócio.

Sistemas Externos Integrados:

API WhatsApp (Meta): Dispara notificações automáticas de confirmação de conta, lembretes de chamadas e alertas operacionais.

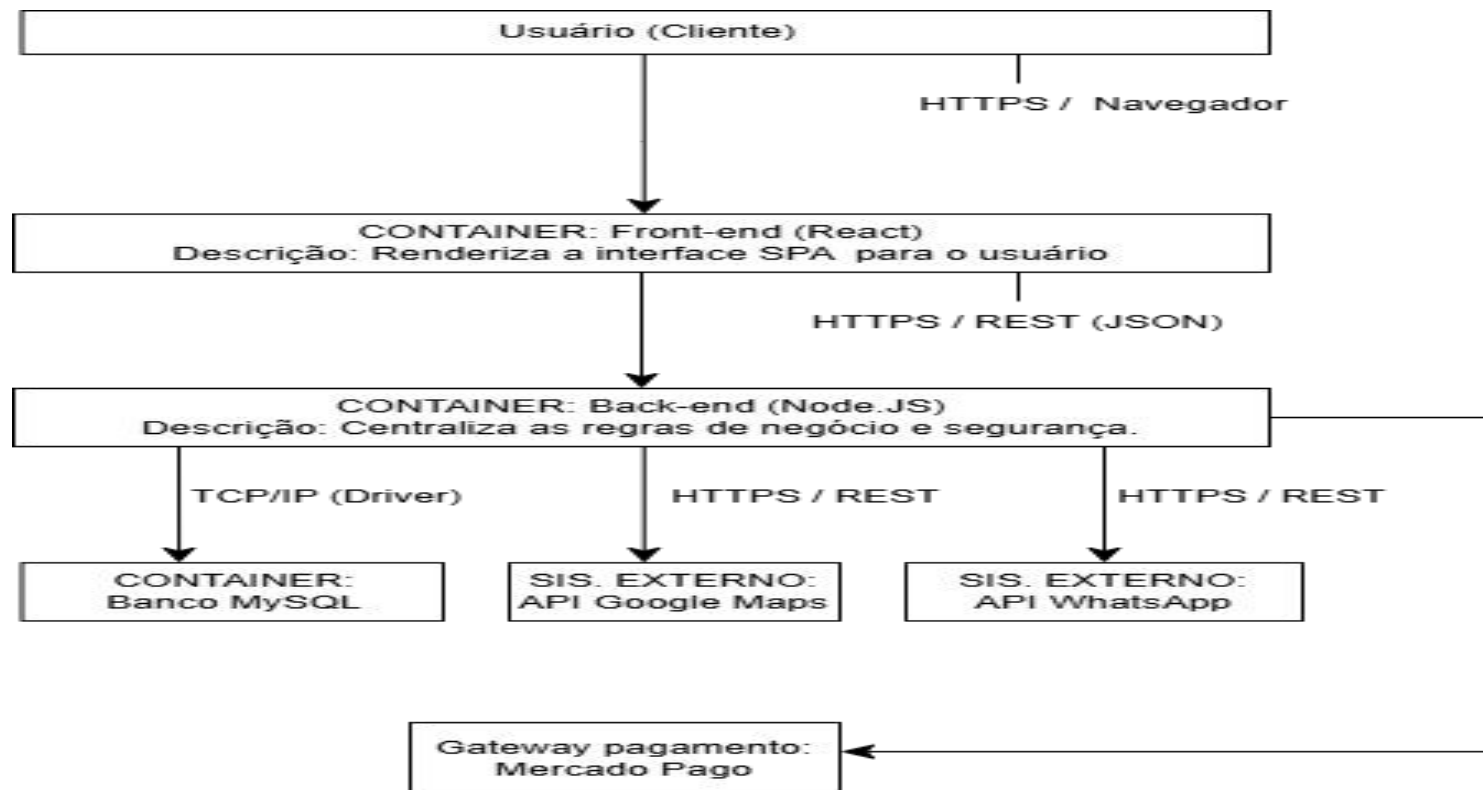
API Mapas (Google Maps): Utilizada para fins de validação geográfica ou localização regional dos profissionais.

Gateway de Pagamento (Mercado Pago): Instituição financeira responsável por processar as cobranças automáticas via Pix ou Cartão de Crédito e validar o status do pagamento.

NÍVEL 2: DIAGRAMA DE CONTAINERS (C2)

O Diagrama de Containers mostra como o Tele Severino foi dividido internamente para funcionar na prática. Nele, o sistema é separado em partes como front-end, back-end e banco de dados, deixando mais claro o papel de cada componente e a forma como eles se comunicam. Esse diagrama ajuda a entender a estrutura da aplicação e as tecnologias usadas em cada camada, sem entrar no nível de código.

2. Figura 2: Diagrama de Containers (C2) detalhando tecnologias.



graph TD

__Usuario[Usuário / Cliente]

__subgraph Sistema [Escopo do Sistema]

____ Front [Container: Front-end
 <i>React / SPA</i>
 <size:10>Renderiza a interface SPA para o usuário</size>]

____ Back [Container: Back-end
 <i>Node.js / Express ou NestJS</i>
 <size:10>Centraliza as regras de negócio e segurança</size>]

____ MySQL [(Container: Banco de Dados
 <i>MySQL</i>
 <size:10>Garante a consistência e persistência dos dados</size>)]

____ end

____ subgraph Externo [Sistemas Externos]

____ Maps [Sistema Externo: API Mapas
 <i>Google Maps</i>]

____ Zap [Sistema Externo: API WhatsApp
 <i>Meta</i>]

____ Pago [Sistema Externo: Gateway Pagamento
 <i>Mercado Pago</i>]

____ end

____ %% Conexões lógicas e seguras

____ Usuario -->|HTTPS / Navegador| Front

____ Front -->|HTTPS / REST JSON| Back

Back -->|TCP/IP / Driver ORM| MySQL

Back -->|HTTPS / REST| Maps

Back -->|HTTPS / REST| Zap

Back -->|HTTPS / REST| Pago

%% Estilos do C4

style Front fill: #2471A3, stroke: #1F618D, stroke-width:2px, color: #fff

style Back fill: #2471A3, stroke: #1F618D, stroke-width:2px, color: #fff

style MySQL fill: #2E4053, stroke: #283747, stroke-width:2px, color: #fff

style Maps fill: #7F8C8D, stroke: #707B7C, stroke-width:2px, color: #fff

style Zap fill: #7F8C8D, stroke: #707B7C, stroke-width:2px, color: #fff

style Pago fill: #7F8C8D, stroke: #707B7C, stroke-width:2px, color: #fff

Relatório Técnico e Justificativas (C2):

Front-end (React): Escolhido para a criação de uma Single Page Application (SPA) dinâmica e de alta performance. O React permite a componentização da interface e uma renderização ágil em tempo real, garantindo uma experiência fluida para clientes e profissionais em dispositivos móveis e web.

Back-end (Node.js): Selecionado devido à sua arquitetura orientada a eventos e I/O não-bloqueante. O Node.js lida de forma extremamente eficiente com requisições assíncronas simultâneas, característica vital para o controle de concorrência e cronometragem de chamadas faturadas por minuto.

Banco de Dados (MySQL): Banco de dados relacional (SGBD) adotado para garantir a consistência rigorosa das informações (propriedades ACID). É fundamental para assegurar a integridade dos históricos de ligações, saldos financeiros e dados cadastrais sensíveis.

Segurança e Protocolos de Rede: A arquitetura foi desenhada seguindo as melhores práticas de segurança de infraestrutura. O contêiner de Front-end comunica-se exclusivamente com o Back-end através do protocolo HTTPS / REST (JSON). O Back-end em Node.js isola e protege o banco de dados MySQL do acesso externo direto, comunicando-se com ele em rede interna via Driver SQL/TCP. Além disso, o Back-end atua como uma barreira segura para consumir as APIs externas de pagamento, mapas e mensageria, prevenindo a exposição de chaves privadas no cliente.

NÍVEL 3: DIAGRAMA DE COMPONENTES (C3)

O Diagrama de Componentes mostra como o back-end do Tele Severino foi organizado por dentro. Ele separa o sistema em módulos menores, como rotas, controllers, serviços e acesso ao banco de dados, deixando mais claro o papel de cada parte no processamento das requisições. A ideia é mostrar como o código se conecta internamente para fazer o sistema funcionar de forma organizada.

Figura 3: Diagrama de Componentes (C3) do Back-end Node.js.Código-fonte do Diagrama

Container:Front-end (React)



CONTAINER: BACK-END (NODE.JS / EXPRESS ou NESTJS)

1.Routes & Controllers (Componente)
Responsabilidade: Escuta os endpoints HTTP e direciona a requisição

Chamadas de Funções / Métodos

2. Business Services(Componente)
Responsabilidade: Executa as regras de negócio e integrações.

Chamadas de Métodos

3. Data Models / Repositories
(Componente - Prisma / Sequelize)
Responsabilidade: Mapeia tabelas e executa queries no banco.

SQL (Driver / ORM)

Container: MySQL

4. Integration Services / Gateways
Responsabilidade: Consome as APIs externas (Axios/Fetch)

HTTPS / REST

APIs Externas
(Maps, WhatsApp, M. Pago)

graph TD

 Front [Container: Front-end
 <i>React / SPA</i>]

 subgraph Backend [Container: Back-end - Node.js]

 Routes[1. Routes & Controllers
 <i>Componente</i>
 <size:10>Escuta os endpoints HTTP e direciona a requisição</size>]

 Services [2. Business Services
 <i>Componente</i>
 <size:10>Executa as regras de negócio e integrações</size>]

 Models [3. Data Models / Repositories
 <i>Componente - Prisma / Sequelize</i>
 <size:10>Mapeia tabelas e executa queries no banco</size>]

 Gateways [4. Integration Services / Gateways
 <i>Componente - Axios / Fetch</i>
 <size:10>Consome as APIs externas</size>]

 end

 MySQL [(Container: MySQL)]

 APIs [APIs Externas
 <i>Maps, WhatsApp, M. Pago</i>]

%% Fluxo de Conexões do seu desenho

Front -->|HTTPS / REST| Routes

Routes -->|Chamadas de Funções / Métodos| Services

Services -->|Chamadas de Métodos| Models

Services -->|Chamadas de Métodos| Gateways

Models -->|SQL Driver / ORM| MySQL

Gateways -->|HTTPS / REST| APIs

%% Estilos visuais limpos

style Backend fill:#ffffff,stroke:#333,stroke-width:2px

style Front fill: #2471A3, stroke: #1F618D, stroke-width:1px, color:#fff

style Routes fill: #ffffff, stroke: #333, stroke-width:1px

style Services fill: #ffffff, stroke: #333, stroke-width:1px

style Models fill: #ffffff, stroke: #333, stroke-width:1px

style Gateways fill: #ffffff, stroke:#333, stroke-width:1px

style MySQL fill: #2E4053, stroke: #283747, stroke-width:1px, color:#fff

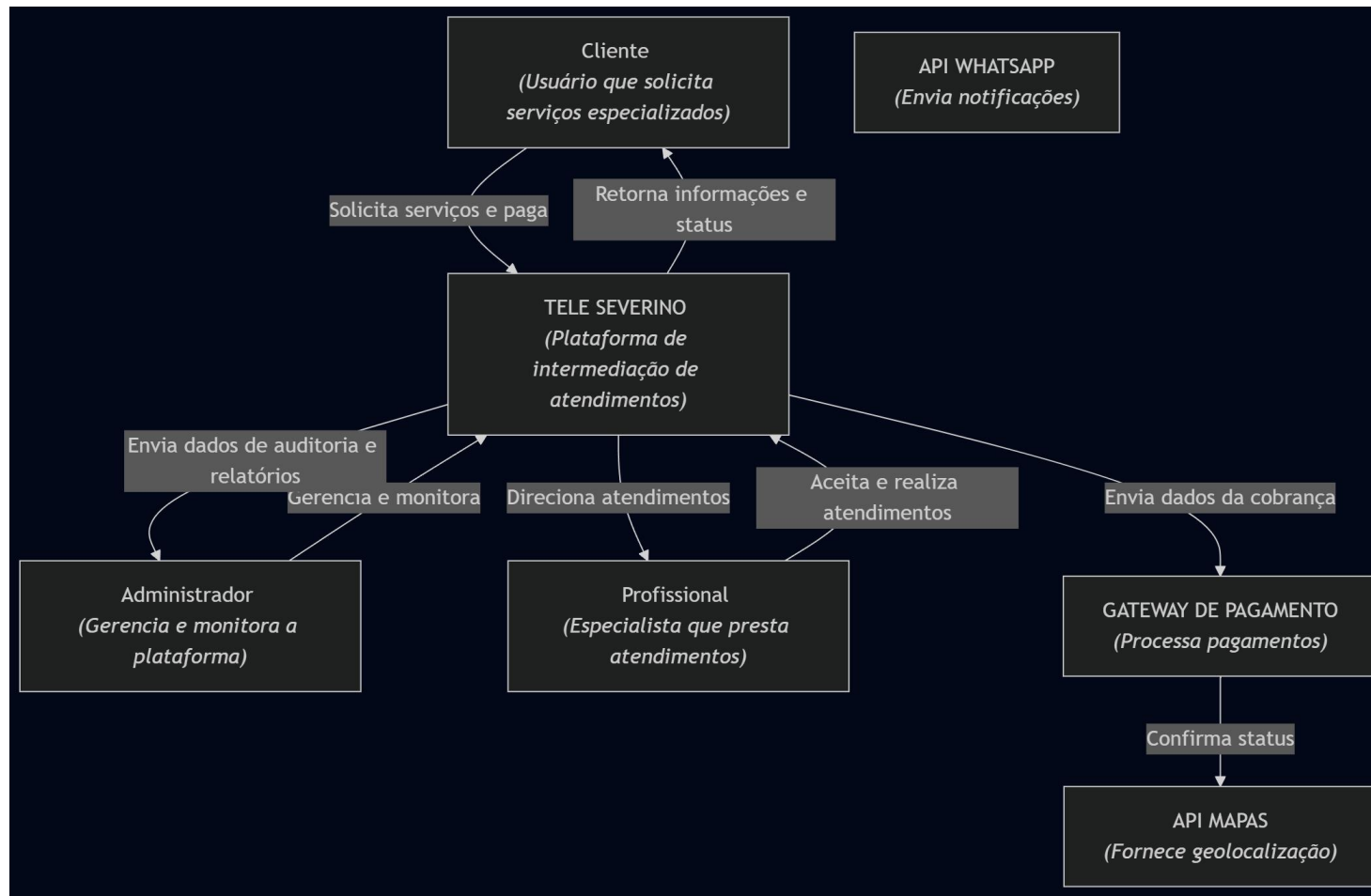
style APIs fill: #7F8C8D, stroke: #707B7C, stroke-width:1px, color:#fff

Relatório Técnico dos Componentes (C3):

1. Routes & Controllers: Camada de interface de rede do Back-end. É responsável por expor os endpoints públicos da API, interceptar as chamadas HTTP enviadas pelo React, efetuar a validação prévia dos dados de entrada (sanitização com Zod/Joi) e despachar a requisição para a camada correspondente.
2. Business Services: O núcleo lógico do sistema. Centraliza todas as regras de negócio do Tele Severino, tais como: verificação de disponibilidade do profissional, cálculo do valor final do atendimento baseado nos minutos cronometrados e aplicação de regras de cupons ou taxas da plataforma.
3. Data Models / Repositories: Camada de persistência que abstrai o banco de dados utilizando uma ORM (como Prisma ou Sequelize). Transforma as instruções Javascript do Node.js em comandos SQL válidos para o MySQL de forma automatizada, blindando a aplicação contra ataques de SQL Injection.
4. Integration Services / Gateways: Módulo isolado especializado na comunicação de saída do servidor. Ele empacota os dados e gerencia os tokens de autenticação necessários para consumir os endpoints do Mercado Pago, WhatsApp e Google Maps, garantindo que o restante do Back-end não precise conhecer os detalhes internos dessas APIs terceiras.

AS CODE

NÍVEL 1: DIAGRAMA DE CONTEXTO (C1)



graph TD

%% Atores e Sistemas (Conforme imagem)

Cliente["Cliente
(Usuário que solicita
serviços especializados)"]

Sistema["TELE SEVERINO
(Plataforma de intermediação de atendimentos)"]

Admin["Administrador
(Gerencia e monitora a plataforma)"]

Profissional["Profissional
(Especialista que presta atendimentos)"]

Gateway["GATEWAY DE PAGAMENTO
(Processa pagamentos)"]

Mapas["API MAPAS
(Fornece geolocalização)"]

Zap["API WHATSAPP
(Envia notificações)"]

%% Relacionamentos e Fluxos Exatos da Imagem

Cliente -->|Solicita serviços e paga| Sistema

Sistema -->|Retorna informações e status| Cliente

Admin -->|Gerencia e monitora| Sistema

Sistema -->|Envia dados de auditoria e relatórios| Admin

Sistema -->|Direciona atendimentos| Profissional

Profissional -->|Aceita e realiza atendimentos| Sistema

Sistema -->|Envia dados da cobrança| Gateway

Gateway -->|Confirma status| Mapas

%% Estilização escura padrão para aproximar do visual da imagem

style Cliente fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Sistema fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Admin fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

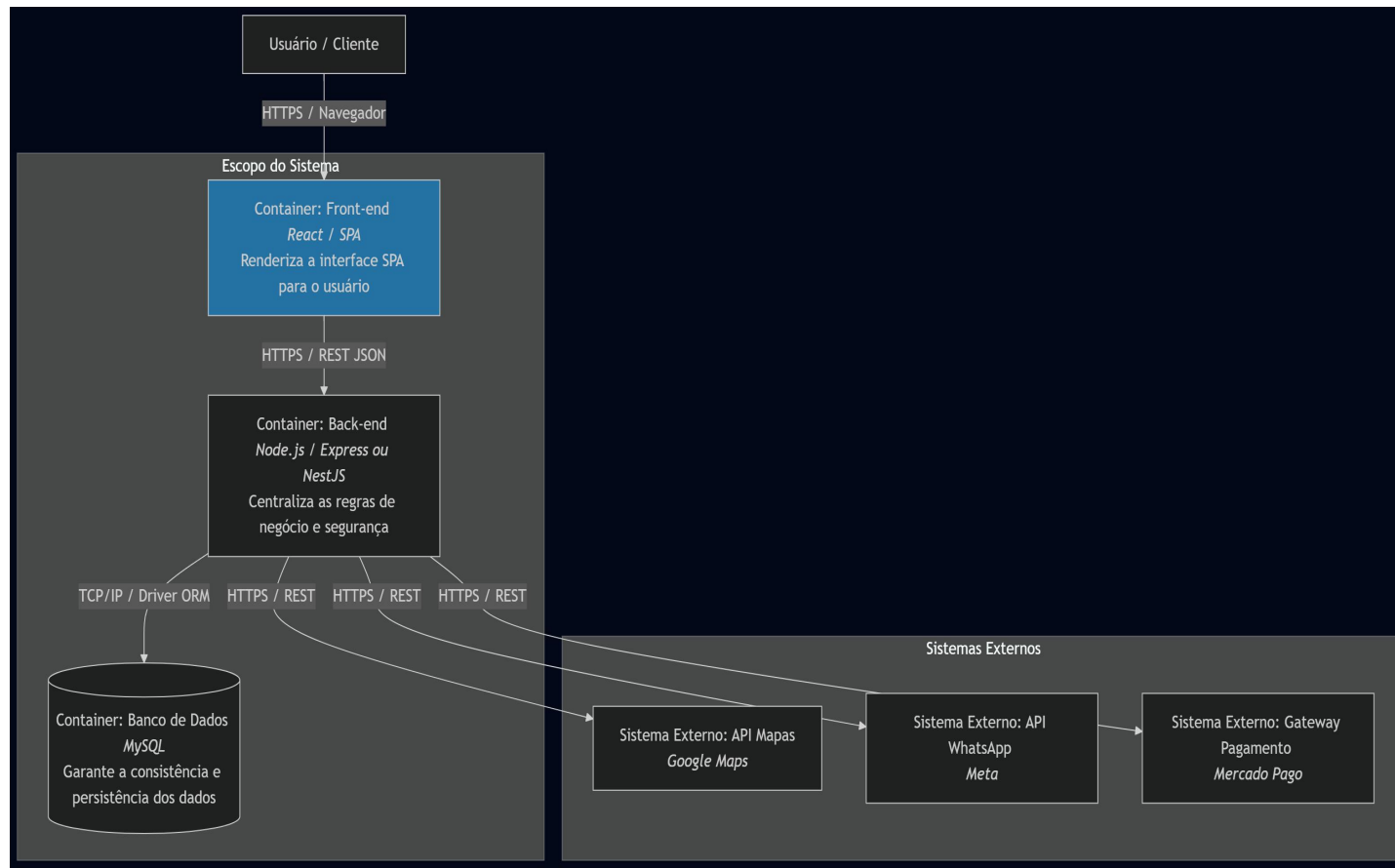
style Profissional fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Gateway fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Mapas fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Zap fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

NÍVEL 2: DIAGRAMA DE CONTAINERS (C2)



graph TD

%% Nós externos ao topo

Usuario["Usuário / Cliente"]

%% Primeiro Subgráfico (Escopo do Sistema)

subgraph Escopo["Escopo do Sistema"]

Front["Container: Front-end
React / SPA

Renderiza a interface SPA
para o usuário"]

Back["Container: Back-end
Node.js / Express ou NestJS

Centraliza as regras de
negócio e
segurança"]

MySQL["Container: Banco de Dados
MySQL

Garante a consistência e
persistência dos dados"]

end

%% Segundo Subgráfico (Sistemas Externos)

subgraph Externos["Sistemas Externos"]

Maps["Sistema Externo: API Mapas
Google Maps"]
Zap["Sistema Externo: API WhatsApp
Meta"]
Pago["Sistema Externo: Gateway Pagamento
Mercado Pago"]
end

%% Relacionamentos e Textos das Linhas de Cima para Baixo

Usuario -->|HTTPS / Navegador| Front

Front -->|HTTPS / REST JSON| Back

%% Conexões saindo do Backend para a Base e Sistemas Externos

Back -->|TCP/IP / Driver ORM| MySQL

Back -->|HTTPS / REST| Maps

Back -->|HTTPS / REST| Zap

Back -->|HTTPS / REST| Pago

%% Estilização escura e azul idêntica à imagem fornecida

style Escopo fill:#222,stroke:#555,stroke-width:1px,color:#fff

style Externos fill:#222,stroke:#555,stroke-width:1px,color:#fff

style Usuario fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Front fill:#1d70b8,stroke:#005ea5,stroke-width:2px,color:#fff

style Back fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

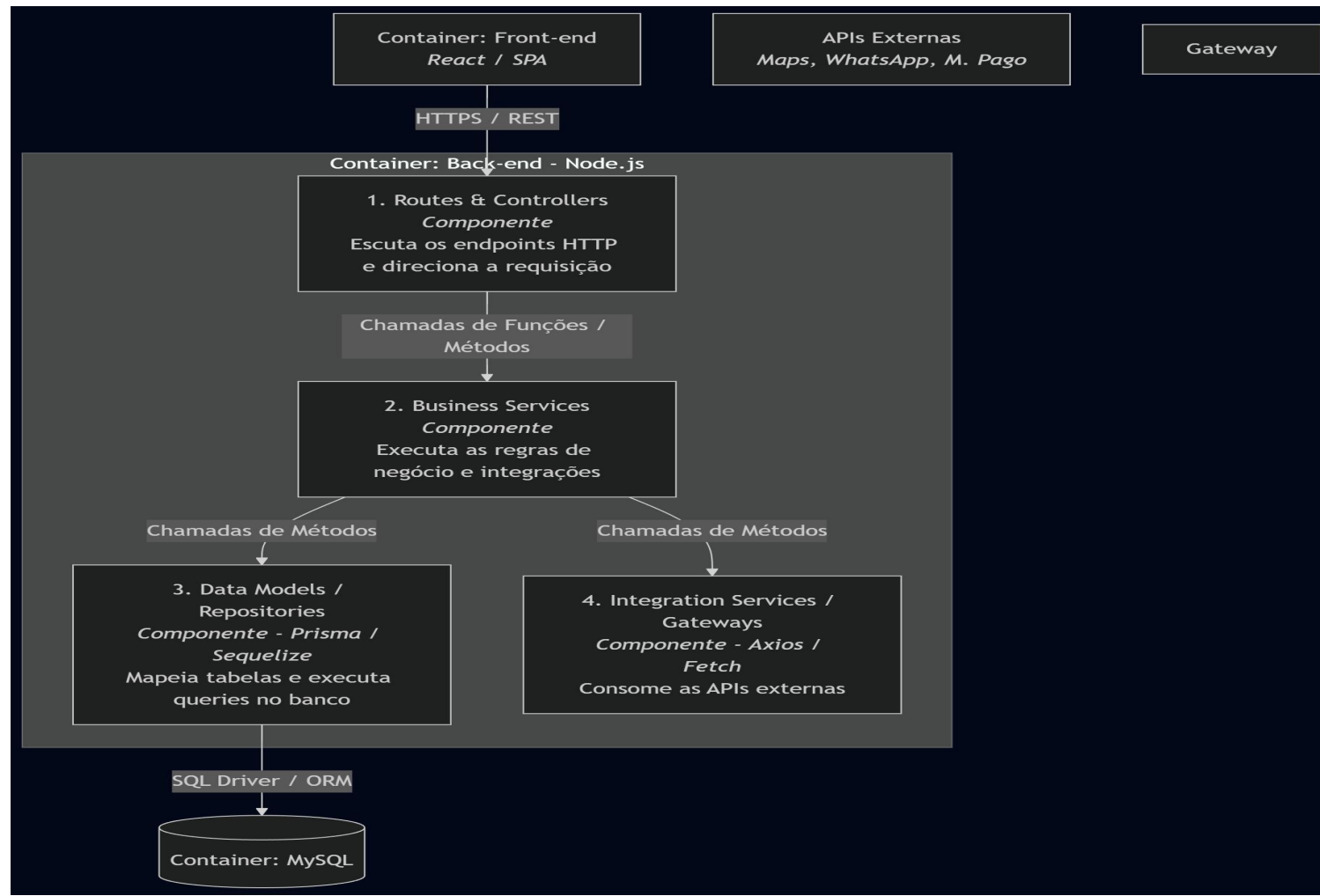
style MySQL fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Maps fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Zap fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Pago fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

NÍVEL 3: DIAGRAMA DE COMPONENTES (C3)



graph TD

%% Nós externos ao topo

Front["Container: Front-end
React / SPA"]

APIs["APIs Externas
Maps, WhatsApp, M. Pago"]

Gateway["Gateway"]

%% Subgráfico Central (Container Back-end)

subgraph Backend["Container: Back-end - Node.js"]

Routes["1. Routes & Controllers
Componente
Escuta os endpoints HTTP
e direciona a requisição"]

Services["2. Business Services
Componente
Executa as regras de
negócio e integrações"]

Models["3. Data Models /
Repositories
Componente - Prisma /
Sequelize
Mapeia tabelas e executa
queries no banco"]

Gateways["4. Integration Services /
Gateways
Componente - Axios /
Fetch
Consome as APIs
externas"]

end

%% Nó externo abaixo

MySQL["Container: MySQL"]

%% Relacionamentos e Fluxos Exatos da Imagem

Front -->|HTTPS / REST| Routes

Routes -->|Chamadas de Funções / Metodos| Services

Services -->|Chamadas de Métodos| Models

Services -->|Chamadas de Métodos| Gateways

Models -->|SQL Driver / ORM| MySQL

%% Estilização para manter a identidade visual escura da imagem

style Backend fill:#222,stroke:#555,stroke-width:1px,color:#fff

style Front fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style APIs fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Gateway fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Routes fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Services fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Models fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style Gateways fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

style MySQL fill:#1f1f1f,stroke:#777,stroke-width:1px,color:#fff

Nota de Implementação: Os diagramas C4 deste documento foram gerados via código utilizando a sintaxe declarativa do **Mermaid**. Essa abordagem foi adotada para garantir que a documentação da arquitetura do Tele Severino seja versionada diretamente no repositório Git, facilitando manutenções futuras sem a dependência de arquivos binários ou ferramentas externas de desenho.